

Dynamic Collaborative Motion Planning in a Factory Workspace

Tristan Dormans, Tomasz Mizera, Alex Andreescu, Nishan Mistry, Maarten Kooij, Yusuf Serhat Özkan

ABSTRACT

The use of cobots (collaborative robots) is rising significantly in the smart manufacturing industry, being able to perform repeated tasks alongside human workers¹. Following these developments, there is a need to ensure that the cobots are able to accurately and reliably perform motion planning in a safe and controlled manner². We compare intrinsically optimized sampling-based algorithms (RRT*, PRM*) with non-optimal algorithms (RRT, PRM) that have been post-processed using shortcutting. This allows for optimization techniques to be compared directly. In addition, we propose a closed-loop planning and execution strategy to address practical execution constraints commonly encountered in industrial robot controllers, such as the lack of trajectory interruption or cancellation. By executing motion plans incrementally and replanning between short execution segments, the system is able to react to dynamic changes in the environment while maintaining predictable behavior. Experiment results show that non-optimal planners combined with shortcutting slightly outperform intrinsically optimized planners in terms of planning time and trajectory complexity across static and dynamic environments, while time-sliced execution provides a practical trade-off between safety and execution efficiency in collaborative settings.

Contents

1	Introduction & Problem statement	1
2	Background	2
3	Methods	2
3.1	Motion Planning	2
3.2	Preparation	2
4	Implementation	3
4.1	Environment Setup	3
4.2	Development	3
5	Experiment	4
5.1	Experimental Setup	4
5.2	Workspace Configurations	4
5.3	Evaluation Metrics	5
5.4	Results and Success Rate Analysis	5
6	Results	6
7	Discussion	6
	References	7

1 Introduction & Problem statement

The manufacturing industry is rapidly becoming a field where collaborative robots (cobots) are increasingly employed to work alongside humans¹. They bring advantages to the field by improving flexibility, reducing cost, and boosting safety in production environments. However, from the industrial point of view, they are often described as slow, not able to provide sufficient payload capacities, and often only used to save the costs of protecting humans from robots².

The problem with collaborative robots arises when they work in spaces alongside humans. Motion planners must be able to react to unpredictable human behavior, maintain safe distances, avoid unnecessary path variability for safety, and make trajectory planners adapt appropriately to dynamic changes³. Robots must be able to respond in real-time to minimize injury risks. As humans also prefer to work with robots that are more adaptive to their environment⁴.

1	PRM	Asymptotic optimality
2	RRT	Asymptotic optimality
3	PRM	Shortcutting (post-processing)
4	RRT	Shortcutting (post-processing)

Table 1. Two sampling-based algorithms and two optimization methods tested in combination

We design a dynamic motion planning and execution system that is fast, collision-free, safe, dynamically adapts to unpredictable changes in the environment and is a collaborative robot arm that works with humans and ensures smooth coexistence. Hence we investigate: "How can various optimization algorithms be compared for robotic arm motion in static and dynamic environments?"

2 Background

Motion planning

Motion planning is an essential tool to find the lowest cost trajectory of a robot, all while avoiding any obstacles present in the environment. There are several types of motion planning algorithms that differ in implementation, performance, and the environment in which they are best suited⁵.

Sampling-based algorithms randomly sample the robot's configuration and produce a graph/map that is followed to produce a solution. These algorithms are probabilistically complete, meaning that a solution is likely to be found after a high run time⁶. Several sampling-based algorithms have been used in robot arm scenarios, such as RRT (rapidly exploring random tree)⁷⁻⁹, and PRM (probabilistic roadmap)^{10,11}. PRM is most commonly used in static environments, given that it is a multi-query planner. As a result, it is expected to have a high-performance cost in dynamic environments, such as the ones in which cobots operate⁵. RRT, on the other hand, is better suited for single-query problems. It is often quick at finding solutions, but they are often suboptimal and long⁷.

Sampling-based algorithms may be optimized to converge to the trajectories with the lowest cost, finding paths that are shorter than those found through regular sample-based algorithms. Both PRM and RRT have optimization-focused versions, PRM* and RRT*. While achieving smoother and shorter paths, these algorithms are often very computationally heavy and take longer than their non-optimized counterparts due to added overhead^{12,13}.

Other motion planning algorithms include deep-reinforcement learning (DRL) algorithms. These algorithms utilize policies that generate actions for the cobot (waypoints, velocities, and torques) to achieve a reward⁵. The reward refines the cobot's decision-making, with this process iterating until the target pose is achieved. Examples of DRL's include Deep Deterministic Policy Gradient¹⁴, Twin Delayed DDPG¹⁵, and Soft-Actor Critic¹⁶. DRL algorithms are often significantly faster than sampling-based or optimization-based algorithms, having a high execution speed, and being able to handle complex environments⁵. However, a DRL may perform very poorly on an environment it isn't trained on, making it less reliable and introducing safety concerns¹⁴.

Shortcutting

Shortcutting is a technique used for path post-processing and is applied on paths that are suboptimal or too long. Shortcutting is achieved by picking points along the trajectory and connecting them to check if a collision exists between them. If none is found, this new path replaces the old one¹⁷. There exist several types of shortcutting algorithms that have been used to post-process trajectories for robotic arms. These include Random Shortcutting¹⁸, forward-loop shortcutting¹⁹, and backward-loop shortcutting²⁰. Hence, the performance of applied shortcutting can directly be compared with optimized versions of algorithms, as both use different implementations to optimize sampling-based algorithms⁵.

3 Methods

3.1 Motion Planning

For every waypoint we compare four different optimized planners based on the total length of the trajectory they produce (our cost function). Two things are compared. Firstly, we compare intrinsic optimization with post-processing through shortcutting, and apply this comparison to two different sampling-based algorithms (PRM and RRT). One is hence chosen as the final algorithm and optimization method for that specific waypoint. See table 1: [1](#)

3.2 Preparation

For our lab setup, we used a KUKA LBR iiwa 14 robot together with KUKA Sunrise Workbench. We relied on existing Java code on the workbench from the project CoRoSect, which enabled communication with the robot via ROS from the lab

computer.

After the CoRoSect project, the robot arm had a soft pneumatic gripper mounted on an extension rail. We had to remove it and install our OnRobot RG6 gripper instead. Next, we had to update the URDF describing the robot in order to include the new gripper in MoveIt and RViz. Adding the camera mounted on the new end effector turned out to be problematic, as we had to manually rotate it in the URDF file.

The lab computer was preinstalled with Ubuntu 20.04 and equipped with 128 GB RAM, an NVIDIA RTX 4090 GPU, and an AMD Ryzen 9 7950X CPU. We used the 3D models of the robot and its environment provided by the CoRoSect project.

An Intel RealSense D435 depth camera was mounted approximately 80 centimeters above the robot (in its default pose), with the camera oriented downwards.

After changing the end effector in the simulation, the robot stopped communicating correctly with the lab computer. Therefore, we copied the workspace from the CoRoSect project and used it as a base for introducing the necessary modifications to adapt the robot model and other parameters to our requirements.

For simulation setup, we made a Docker image of the environment working with GUI on both Windows (via WSL) and Linux so that all members can work in the same reproducible environment.

4 Implementation

4.1 Environment Setup

In order to help our workflow, we needed to simulate the lab environment on our local machines. The software only works on the Ubuntu 20.04 operating system. Most of our team was using Windows, which had NVIDIA Isaac Sim as an alternative for robotics simulation. Unfortunately, the application cost a lot of computing power and resulted in poor performance on our machines.²¹

To solve this issue, we switched to a Linux-based setup by installing Ubuntu 22.04, either directly or through virtual machine software like Oracle's VirtualBox, Broadcom VMware, or by using Docker. The software we needed to install was ROS Noetic, which provides libraries and tools for building robot applications and managing sensors, actuators, and communication. ROS provides a modular framework for robotics applications, enabling distributed processes (nodes) to communicate through standardized message-passing interfaces. ROS was used to integrate motion planning, robot control, perception, and visualization into a coherent pipeline and allowed for individual components to be developed and tested independently.

Motion planning was done through MoveIt 1.1.16, a ROS-based framework that provides collision checking, kinematic modeling, and access to various sampling-based planning algorithms. This allowed us to use motion planning at a high level while the framework handled the kinematics and collision checking in the background.

Configuration of the robot model and motion planning environment was done using the MoveIt Setup Assistant. The URDF description of the KUKA iiwa 14 was loaded, after which the kinematic chain, planning groups, end-effector configuration, and collision and self-collision matrices were generated. While collision matrices can be generated automatically, planning groups and end-effector settings require manual configuration.

During manipulation tasks, the dimensions of the carried object (slider) were incorporated into the planning scene as an attached collision object, ensuring collision-free motion while the robot was holding it. The attachment state of the slider was determined based on feedback provided by the visual servoing project group. The system subscribed to a dedicated ROS topic to monitor whether the slider was currently grasped and updated the planning scene accordingly.

Visualization and debugging were performed using RViz 1.14.26, giving us real-time feedback of the robot state, planned trajectories, and the planning scene. This software was used to inspect motion plans, verify collision objects, and monitor the execution of trajectories before execution on the physical robot.

For a physics-based simulation of the robot and testing its environment and the Octomap, we installed Gazebo. This enabled us to set up a virtual depth camera, test motion planning behavior in a dynamic setting and observe interactions with simulated obstacles.

In order to maintain a consistent development environment across machines, we created a Docker image containing the full ROS Noetic and MoveIt setup, including all project dependencies. The Docker environment was configured with hardware acceleration support to optimize RViz and Gazebo performance. GitHub was used to ensure our codebase was consistent between all machines.

4.2 Development

After configuring the robot model and planning environment, motion planning was used using planners provided by OMPL through the MoveIt interface. For each motion request, multiple planners as mentioned previously, were run for a maximum planning time of 10 seconds and 3 planning attempts per query. For each successful plan, the trajectory was evaluated by a cost

function based on total joint-space path length. The path with the smallest overall joint movement is selected in order to avoid overcomplicated and unpredictable trajectories.

Once a trajectory is selected, it is sent to the queue for execution on the physical robot. During testing, we realized that once execution begins, the robot continues to follow the queued trajectory unless the controller application is restarted, and there is no way to interrupt, cancel, or pause motion through software. This results in the system lacking a reliable fallback for immediately stopping execution when unexpected obstacles enter the workspace. This makes conventional approaches for dynamic obstacle avoidance, like continuous online replanning or reactive stopping, impractical. In a collaborative environment, this poses major safety concerns, since if a new obstacle is added and detected in the planning scene, the robot continues executing the previously queued trajectory to completion.

To mitigate this problem, we opted to avoid long, uninterrupted trajectory execution. Instead of executing an entire trajectory at once, the system only executes a short portion of the planned motion at a time. After the initial trajectory is generated, we send only the first 1 second of motion to the robot execution queue. The remainder of the trajectory is discarded and once the segment execution finishes, the robot state is updated and a new planning cycle is started from the robot's current position. This process is repeated until the target pose is reached within a predefined tolerance. For joint-space targets, the goal is considered reached when the mean absolute joint error falls below 0.08 radians, and for Cartesian targets, the end-effector position error must be below 0.03 m. A maximum execution time of 60 seconds per motion segment is enforced to prevent indefinite replanning in failure cases.

Time-sliced execution introduces a trade-off between safety and execution speed. Shorter slicing intervals increase the responsiveness to dynamic changes in the environment, allowing the robot to react quicker to new or moving obstacles. However, they also increase planning frequency and computational overhead, reducing overall execution efficiency. Longer slicing intervals improve execution efficiency but reduce responsiveness, increasing the risk of delayed reactions. The slicing duration was agreed to be 1 second, since testing showed that 0.5 seconds is too slow, and 1.5 seconds is too unsafe.

To account for dynamic obstacle representation, we tried using OctoMap-based occupancy mapping as an input to MoveIt's planning scene. The point cloud was transformed into the planning frame, filtered, and fused into an OctoMap, which was then used for collision checking during planning. In practice, newly added obstacles appeared in the occupancy map within ~ 1.5 -2 seconds, which isn't that bad, but clearing an obstacle after it has already left the scene takes ~ 10 -15 seconds. This unfortunately makes the OctoMap unreliable and hard to use and was therefore not used in the final build.

This execution strategy allows for safe, incremental motion in dynamic environments while remaining compatible with the constraints of the robot controller.

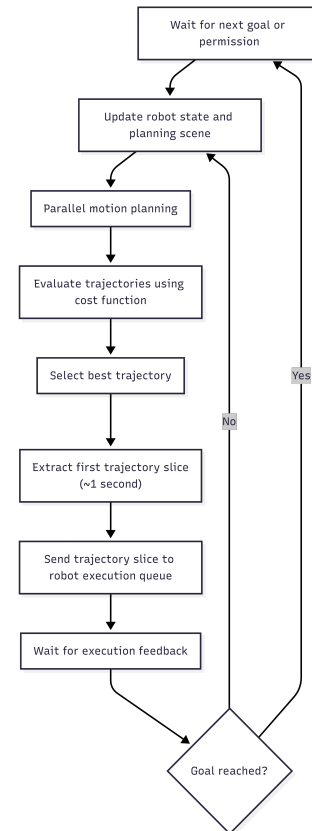


Figure 1. Closed-loop planning and execution cycle.

5 Experiment

To evaluate the performance of the selected motion planners, we conducted a series of benchmarks¹³ across three distinct workspace configurations: *no_obstacles*, *static_obstacles*, and *dynamic_obstacles*. These environments were designed to simulate various stages of a factory floor operation, from an empty workspace to one shared with moving human workers.

5.1 Experimental Setup

The benchmarking process involved a total of 9,000 planning attempts. To ensure statistical significance, we performed 150 runs per environment for each of the four planners: PRM*, RRT*, PRM+Shortcut, and RRT+Shortcut. Each planner was granted a maximum of 10 seconds per query to find a valid solution, with up to 3 attempts allowed per motion request.

5.2 Workspace Configurations

Three environments were designed to represent typical industrial scenarios of increasing complexity:

1. **No Obstacles:** A baseline environment with an empty workspace, used to measure the fundamental overhead of each algorithm.

2. **Static Obstacles:** The workspace included fixed collision geometries representing machinery and safety fencing. This required the planners to navigate around non-linear constraints.
3. **Dynamic Obstacles:** This environment simulated a collaborative workspace where an obstacle entered the robot's intended path after the initial plan was generated. This required the system to trigger replanning cycles as the occupancy map updated, ensuring the robot adapted its trajectory in real-time to maintain safety.

5.3 Evaluation Metrics

The planners were compared based on the following three criteria:

- **Planning Time:** The Wall-clock time required to generate a valid trajectory, measured in seconds.
- **Waypoint Count:** The number of discrete configurations (states) in the resulting path. A lower waypoint count is generally preferred for smoother robot execution.
- **Path Length:** Calculated as the Euclidean distance in Joint Space (Configuration Space). While distance in a factory is typically measured in meters, motion planning for articulated manipulators is performed in C-space, where each coordinate represents a joint angle. If the robot moves from configuration q_1 to q_2 , the distance d is calculated as:

$$d(q_1, q_2) = \sqrt{\sum_{i=1}^n (q_{1,i} - q_{2,i})^2}$$

where q_i is the angle of the i -th joint in radians. Using joint-space distance instead of Cartesian distance (meters) ensures that the metric accounts for the motion of the entire kinematic chain rather than just the end-effector.

5.4 Results and Success Rate Analysis

Across all 9,000 planning attempts, all four algorithms achieved a 100% success rate. While sampling-based planners are inherently stochastic, this perfect success rate can be attributed to two factors. First, the 10-second timeout per query provided ample time for the algorithms to explore the configuration space, exploiting the probabilistic completeness of RRT and PRM variants. Second, all target waypoints were pre-validated to be within the kinematic reach and joint limits of the iiwa 14.

Table 2. Overall comparison of planner performance averaged across all environments. All planners achieved a 100% success rate.

Planner	Success Rate (%)	Avg. Time (s)	Avg. Path Length (rad)	Avg. Waypoints
PRM*	100.0	0.239	1.890	23.40
RRT*	100.0	0.180	1.888	23.40
PRM + Shortcut	100.0	0.171	1.740	2.30
RRT + Shortcut	100.0	0.183	1.715	2.28

Table 3. Detailed performance breakdown by environment. Shortcutting methods consistently reduced waypoint counts and path lengths compared to intrinsic optimization methods.

Environment	Planner	Time (s)	Path Length (rad)	Waypoints
No Obstacles	PRM*	0.176	1.565	19.61
	RRT*	0.114	1.565	19.58
	PRM + Shortcut	0.112	1.565	2.03
	RRT + Shortcut	0.131	1.565	2.00
Static Obstacles	PRM*	0.292	2.123	26.06
	RRT*	0.214	2.096	25.73
	PRM + Shortcut	0.210	1.935	2.59
	RRT + Shortcut	0.240	1.943	2.62
Dynamic Obstacles	PRM*	0.250	1.984	24.55
	RRT*	0.212	2.005	24.86
	PRM + Shortcut	0.190	1.722	2.30
	RRT + Shortcut	0.177	1.638	2.23

Table 4. Best-performing planner for each environment based on planning time and path length.

Environment	Fastest Planner	Shortest Path Planner
No Obstacles	PRM + Shortcut (0.112 s)	PRM + Shortcut (1.565 rad)
Static Obstacles	PRM + Shortcut (0.210 s)	PRM + Shortcut (1.935 rad)
Dynamic Obstacles	RRT + Shortcut (0.177 s)	RRT + Shortcut (1.638 rad)

6 Results

The results achieved indicate the performance of each sampling-based planner and optimization algorithm that was analyzed. Firstly, all planners had a success rate of 100% for the coordinates tested, as mentioned above. None of the planners failed to find a solution in any of the three environments tested. We found that PRM+shortcutting yielded the least time in all 3 environments (0.112, 0.210, and 0.190, respectively), while PRM* yielded the highest, even though the same sampling-based algorithm was used. For path length, all planners yielded the same trajectory, and thus each had a path length of 1.565 rad. However, for environments with obstacles, PRM* and RRT* yielded the largest path lengths, respectively, while PRM+Shortcutting and RRT+Shortcutting yielded the lowest. Waypoints were found to be significantly lower for any shortcutting case compared to asynchronously optimized cases.

7 Discussion

The results indicate that shortcutting is very effective at lowering the time taken and reducing path length, indicating that it is better at optimizing the trajectory found compared to asynchronous optimization. Shortcutting produced shorter paths due to creating collision-free connections between waypoints, while even PRM* and RRT* often may include detours or produce jagged paths¹². The only exception to this was in the obstacle-free environment, where the path produced was likely straight, and thus already optimal. However, this environment would not match a real factory situation.

In obstacle-free and static obstacle environments, the time taken by the asynchronous optimization and shortcutting was roughly equal. This is due to the planning time being dominated mostly by the planner itself. In a dynamic environment, however, the reduced number of waypoints results in fewer collision checks and less computational power required, resulting in less time taken to achieve an optimized trajectory¹³.

The reduction in waypoints merely points to the algorithm behind shortcutting, which reduces waypoints by removing intermediate points if a direct path is available. Hence why the number of waypoints reduced significantly⁵.

In dynamic obstacle environments, if the base algorithm was PRM, it took on average more time to find the path. This is due to PRM constructing a predefined roadmap statically. Hence, it takes longer to adapt to dynamic obstacles compared to RRT¹³. Nevertheless, the differences between PRM and RRT did not significantly influence path length in both static and dynamic obstacle environments.

The project was integrated with another project group responsible for fault detection. Within the designed technological

process (slider bin -> press -> fault detection -> output slider bin), inter-group communication was implemented using ROS topics. The motion planning system publishes a boolean signal indicating whether the robot has reached a predefined inspection pose, corresponding to the position at which the fault detection model was trained using input from the camera mounted on the robot. This signal enables the fault detection module to reliably perform visual inspection of the slider while it is positioned in the press.

We suggest not to use outdated versions of Ubuntu, 20.04 for example. Additionally, make sure the computer has an ethernet connection, as the WiFi can be unreliable. Software groups should not work on the computer that directly controls the hardware, in order to maximize the availability of lab time for hardware-focused groups. The Intel RealSense D435 camera's depth sensor should not be used for long-distance object detection, due to high noise. Additionally, outdated versions of ROS and MoveIt should be avoided. Only the official USB extension cable for the ZED camera should be used (third-party extension cables can cause problems with communication). Windows Subsystem for Linux (WSL) should be avoided for development in hardware-related projects of such complexity for compatibility and efficiency (CPU, GPU) reasons. When building with catkin, not every MoveIt package should be build in the src directory. Instead, build only the packages that have been modified. After building with catkin, always source `devel/setup.bash` before running or testing the system. Before starting the project, ensure that the robot can be safely stopped after sending a goal command. Development and execution should be performed using GUI-enabled Docker with hardware acceleration (`gui-docker`).

- We recommend to research first if [OctoMap](#) will work for your use case before using it for the detection of dynamic obstacles. We experienced poor performance of the Octomap, it is slow to remove voxels that are no longer occupied. Other people also reported this problem on [github](#), we have also tried newer ROS2 and MoveIt2 Humble and also observed poor performance. We have observed a delay of at least 9 seconds. (We installed this [humble-tutorial-source](#) and followed this guide to try out the OctoMap in ROS2, [MoveIt2 perception pipeline tutorial](#). Our experience with the OctoMap on bare metal in the lab was similar. Our ad hoc solution to this was to delete all voxels in a fixed interval. The hardware does not seem to be the limiting factor. During the execution of the tutorial, CPU, GPU and RAM were all utilized at at most 30%.
A better alternative for dynamic obstacle detection for ROS1 than OctoMap would be [skimap](#), judging from the occupancy map's higher updating speed in the demonstration video. Unfortunately, we ran out of time before we could get it to work.
- For dynamic obstacle environments that best represent the environment in which a cobot would operate alongside human workers, shortcutting was shown to most effectively optimize the trajectory, producing the shortest paths in the lowest time. However, if the environment is free from objects, these differences are not significant.

Disclaimer that Rviz on ROS1 Noetic performs poorly, the modeled robot environment, stresses the GPU heavily, even when it is relatively simple. The exact same 3D model of the environment in Gazebo is much less taxing (from person experience, 29% GPU usage Gazebo, Rviz 93% GPU usage set at 30 FPS, GTX 1050 Ti)

Conclusion

We developed and evaluated a motion planning framework for a collaborative robot operating in a factory-like environment. The system combines sampling-based motion planning, trajectory post-processing, and incremental execution to address practical challenges commonly encountered in human-robot collaboration, such as dynamic obstacles and limited execution control. By executing only short segments of a planned trajectory and replanning between execution steps, the system is able to remain responsive to changes in the environment while maintaining safe and predictable behavior.

Experiment results showed that non-optimal planners combined with shortcutting slightly outperformed intrinsically optimal planners in terms of planning time, trajectory complexity, and execution smoothness in dynamic obstacle environments, indicating that this algorithm may be favourable for cases with cobots. Time-sliced execution proved effective in dealing with the lack of trajectory interruption. Future work could explore faster environment representations and improved execution control to increase robustness in dynamic human-robot interaction scenarios.

References

1. Patil, S., Vasu, V. & Srinadh, K. S. Advances and perspectives in collaborative robotics: a review of key technologies and emerging trends. *Discov. Mech. Eng.* **2**, DOI: [10.1007/s44245-023-00021-8](#) (2023).
2. Llopis-Albert, C., Rubio, F. & Valero, F. Modelling an industrial robot and its impact on productivity. *Mathematics* **9**, 769, DOI: [10.3390/math9070769](#) (2021).

3. Beschi, M., Faroni, M., Copot, C. & Pedrocchi, N. How motion planning affects human factors in human-robot collaboration. *IFAC-PapersOnLine* **53**, 744–749, DOI: [10.1016/j.ifacol.2021.04.167](https://doi.org/10.1016/j.ifacol.2021.04.167) (2020).
4. Lasota, P. A. & Shah, J. A. Analyzing the effects of human-aware motion planning on close-proximity human–robot collaboration. *Hum. Factors* **57**, 21–33, DOI: [10.1177/0018720814565188](https://doi.org/10.1177/0018720814565188) (2015).
5. Liu, J., Yap, H. J. & Anis. Review on motion planning of robotic manipulator in dynamic environments. *J. Sensors* **2024**, DOI: [10.1155/2024/5969512](https://doi.org/10.1155/2024/5969512) (2024).
6. Zhang, L. *et al.* Motion planning for robotics: A review for sampling-based planners. *Biomim. Intell. Robotics* **5**, 100207, DOI: [10.1016/j.birob.2024.100207](https://doi.org/10.1016/j.birob.2024.100207) (2025).
7. Ding, Z.-J. *et al.* Variable sampling area rrt algorithm based on limiting joint angle for robotic arm path planning. *IEEE Access* DOI: [10.1109/access.2025.3541645](https://doi.org/10.1109/access.2025.3541645) (2025).
8. Li, Y. & Chen, N. Improved rrt algorithm for robotic arm path planning based on reward strategy. *Proc. Inst. Mech. Eng. Part C: J. Mech. Eng. Sci.* **239**, 6983–6995, DOI: [10.1177/09544062251344137](https://doi.org/10.1177/09544062251344137) (2025).
9. Fragkopoulos, C. & Graeser, A. A rrt based path planning algorithm for rehabilitation robots (2026). Available at IEEE Xplore.
10. Jaillet, L. & Siméon, T. A prm-based motion planner for dynamically changing environments. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, DOI: [10.1109/iros.2004.1389625](https://doi.org/10.1109/iros.2004.1389625) (2004).
11. Joao, A. *et al.* Analysis of the trajectory planning of a scara robot using the prm algorithm. In *International Conference on Control, Mechatronics and Automation (ICCMA)*, 66–70, DOI: [10.1109/iccma56665.2022.10011598](https://doi.org/10.1109/iccma56665.2022.10011598) (2022).
12. Jiang, J., You, Y., Shao, Y., Bi, Y. & You, J. Robot path planning based on improved prm for wing-box internal assembly. *Machines* **13**, 952, DOI: [10.3390/machines13100952](https://doi.org/10.3390/machines13100952) (2025).
13. Liu, S. & Liu, P. Benchmarking and optimization of robot motion planning with motion planning pipeline. *The Int. J. Adv. Manuf. Technol.* **118**, 949–961 (2022).
14. Wen, S., Chen, J., Wang, S., Zhang, H. & Hu, X. Path planning of humanoid arm based on deep deterministic policy gradient. In *IEEE International Conference on Robotics and Biomimetics (ROBIO)*, 1755–1760, DOI: [10.1109/robio.2018.8665248](https://doi.org/10.1109/robio.2018.8665248) (2018).
15. Kim, M., Han, D.-K., Park, J.-H. & Kim, J.-S. Motion planning of robot manipulators for a smoother path using a twin delayed deep deterministic policy gradient with hindsight experience replay. *Appl. Sci.* **10**, 575, DOI: [10.3390/app10020575](https://doi.org/10.3390/app10020575) (2020).
16. Wong, C.-C., Chien, S.-Y., Feng, H.-M. & Aoyama, H. Motion planning for dual-arm robot based on soft actor-critic. *IEEE Access* **9**, 26871–26885, DOI: [10.1109/access.2021.3056903](https://doi.org/10.1109/access.2021.3056903) (2021).
17. Huang, P., Shaoul, Y. & Li, J. Benchmarking shortcutting techniques for multi-robot-arm motion planning (2025). ArXiv:2508.05027.
18. Siciliano, B. *Springer handbook of robotics* (Cham Springer International Publishing, 2016).
19. Hsu, D., Latcombe, J.-C. & Sorkin, S. Placing a robot manipulator amid obstacles for optimized execution. In *IEEE International Symposium on Assembly and Task Planning (ISATP)*, DOI: [10.1109/isatp.1999.782972](https://doi.org/10.1109/isatp.1999.782972) (2003).
20. Chen, P. & Hwang, Y. Sandros: a dynamic graph search algorithm for motion planning. *IEEE Transactions on Robotics Autom.* **14**, 390–403, DOI: [10.1109/70.678449](https://doi.org/10.1109/70.678449) (1998).
21. NVIDIA. Isaac sim installation requirements (2024).